Percolation Screening on the Cross-Boundary Shadow Graph

Flyxion
Independent Researcher

November 2026

1 Percolation Screening on the Cross-Boundary Shadow Graph

1.1 Two Graphs With Different Jobs

Let $G = (V, E, r)$ denote the full graph whose vertices are symbols and whose directed edges carry a relation label $r(e)$ drawn from a finite set R (import, possible call, shared type, memory reference, authorized write, and so on). Let $G_{\text{shadow}} = (V, E_{\text{shadow}})$ be the subgraph retaining only the edges (u, v) with $\text{lut}[u] \neq \text{lut}[v]$, that is, the edges that cross a package boundary. The shadow graph is used to measure the structure of boundary crossings. It is not used to search for propagation paths, because deleting within-package edges breaks the composition of the paths that matter: a dependency that calls a public function in one package, which calls an internal helper, which invokes a sensitive operation in a third package, contains two crossing edges joined by an internal one, and in G_{shadow} those two crossings no longer connect. The absence of a witness in G_{shadow} is therefore not a sound negative test for the existence of a path in the software, although the crossing structure of G_{shadow} remains informative about how much boundary-crossing exposure the system has. Witness searches are performed in G, or in G augmented with explicitly justified within-package summary edges whose semantics and provenance are recorded.

1.2 Thresholds and Their Limits

The Erdős–Rényi evolution of $G(n, p)$ [8] supplies the vocabulary for the structural question. The expected degree is $(n-1)p$, with $c = np$ the usual asymptotic parameter, and for fixed c away from one the largest component has $O(\log n)$ vertices with high probability when $c < 1$ and $\beta n + O_p(\sqrt{n})$ vertices when $c > 1$, where β is the positive root of $\beta = 1 - e^{-c\beta}$. At $c = 1$ the largest component has order $n^{2/3}$ within the critical window. The undirected quantities below are computed on the undirected projection $H_{\text{shadow}} = (V, F_{\text{shadow}})$, where $F_{\text{shadow}} = \{\{u, v\} : (u, v) \in E_{\text{shadow}} \text{ or } (v, u) \in E_{\text{shadow}}\}$, so that opposite directed edges and multiple relations between the same pair of symbols collapse into one undirected edge. Its mean degree is $\bar{k} = 2|F_{\text{shadow}}|/|V|$, which plays the role of c only when the graph is modelled as $G(n, p)$. The ratio K is likewise computed on H_{shadow}, while K_{dir} is computed on the directed shadow graph. Mean degree is a poor summary when degrees are heterogeneous, and the Molloy–Reed criterion replaces it by a ratio of degree moments,

$$K = \frac{\langle k^2 \rangle}{\langle k \rangle} = \frac{\sum_{v \in V} \deg(v)^2}{\sum_{v \in V} \deg(v)}.$$

For an admissible sequence of prescribed-degree random graphs satisfying the regularity conditions of Molloy and Reed [3], a limiting ratio $K_n \to K_\infty > 2$ places the ensemble in the supercritical regime, in which a giant component occupies a positive fraction of the vertices. A finite sequence

with $K_n > 2$ at every n need not do so if K_n approaches two, so the statement concerns the limiting ratio strictly above the threshold. This is a single criterion written two ways, since $\sum_v k_v > 0$ gives $K > 2$ if and only if $\sum_v k_v(k_v - 2) > 0$. Heavy tails do not by themselves guarantee the regularity conditions, so the criterion extends the Erdős–Rényi threshold to heterogeneous degree sequences of this regular kind and no further. For a Poisson degree distribution $\langle k^2 \rangle = c + c^2$, so $K = 1 + c$ and the conditions $K > 2$ and $c > 1$ coincide.

The criterion describes typical behaviour in an ensemble, and an individual observed graph need not exhibit that behaviour. A disjoint union of many four-vertex cliques has every degree equal to 3, hence $K = 3 > 2$, while no component has more than four vertices. The union of cliques is an admissible but atypical member of the prescribed-degree ensemble, since a graph drawn by uniform stub matching from this degree sequence would almost surely contain a giant component. The example therefore demonstrates the limitation of applying a typical-case ensemble conclusion deterministically to an individual graph, and it exhibits clustering as one way in which a graph can be atypical. Real dependency graphs may depart from the null through clustering and degree correlations, since hubs such as shared utility libraries could attach to a wide and nonrandom set of neighbours. The ratio K is accordingly a contextual statistic describing branching propensity under a specified model, and it does not certify connectivity of the graph at hand.

Edges in a dependency graph are directed. Write k_v^{in} and k_v^{out} for the in- and out-degrees and assume $\langle k \rangle = \langle k^{\text{in}} \rangle = \langle k^{\text{out}} \rangle$, which holds automatically because every edge contributes one in-degree and one out-degree and which the following formula relies on. Under the corresponding degree-sequence regularity assumptions, and with the same qualification that the limiting ratio must lie strictly above the threshold, the condition for a giant strongly connected component in the directed configuration model is

$$K_{\text{dir}} = \frac{\langle k^{\text{in}} k^{\text{out}} \rangle}{\langle k \rangle} > 1,$$

established rigorously by Cooper and Frieze [5], with an earlier heuristic derivation by Dorogovtsev, Mendes and Samukhin [4]. This answers a narrower question than the one security screening asks. Strong connectivity is not necessary for extensive compromise: a directed chain on n vertices gives its first vertex a forward reach set of size n while every strongly connected component is a single vertex, and a directed star gives its centre access to every other vertex while every product $k_v^{\text{in}} k_v^{\text{out}}$ is zero. A malicious dependency can be a pure source and a sensitive operation a pure sink, so $K_{\text{dir}} > 1$ is not a prerequisite for screening. The operational quantity is the forward reach set of a specified entry vertex, and K_{dir} serves only as background characterization of the degree structure.

1.3 The Null Ensemble

Three decisions define the comparison ensemble: which graphs it contains, which moves are allowed to generate them, and how the samples are drawn and used. The ensemble proposed here consists of graphs that preserve each vertex's in- and out-degree and, when the object of study is the shadow graph, the package labelling of the vertices, with every edge satisfying the constraint $\text{lut}[u] \neq \text{lut}[v]$, that is, every edge crossing a package boundary.

The moves are directed double-edge swaps, in which $(a \to b)$ and $(c \to d)$ become $(a \to d)$ and $(c \to b)$ when no self-loop or duplicate results. If several relation-labelled edges between the same symbols are retained, the null must also fix and report whether it preserves relation counts and whether a duplicate means the same ordered endpoints or the same endpoints and the same label. For shadow graphs the swap must additionally respect the boundary constraint, since an unrestricted swap of an edge from package P to Q with one from Q to P produces edges internal to P and to Q. A stricter variant restricts swaps to edge pairs with the same pair of packages at their

endpoints. That variant preserves each vertex's degree contribution within each package pair, which is considerably more conditioning than preserving only the aggregate number of edges between each pair of packages, and the choice between the two should be reported with the result.

Swaps alone do not give a chain that visits the whole state space. On three vertices with all in- and out-degrees equal to one, the two orientations of a directed cycle cannot be joined, because every pair of the three edges shares a vertex and every swap therefore creates a self-loop or a duplicate. For unconstrained simple digraphs the difficulty is resolved by adding the reorientation of directed 3-cycles, after which the state graph is strongly connected [6, 7]. No corresponding result is assumed here for the constrained chain on shadow graphs with fixed degrees and fixed package labelling. That chain is a restriction of the unconstrained one, and irreducibility does not carry over automatically. If the chain is not irreducible, sampling from the observed graph explores only its reachable component of the state space. This restriction defines a possible conditional null, but a calibrated conditional test additionally requires a specified target law on that component and a justified sampling procedure.

The rank-based Monte Carlo p-value $(1 + \#\{b : T_b \geq T_{\mathrm{obs}}\})/(1 + B)$ is valid when the observed graph and the simulated graphs are exchangeable under the specified null law. Independent draws from that law provide this property when the null hypothesis also assigns that law to the observed graph. Membership in the ensemble alone does not establish exchangeability. The test therefore asks whether the observed graph is typical of the specified law, and it does not ask whether the observed graph is random. A disconnected state space identifies a possible conditional null, but it does not by itself yield a valid conditional p-value, because the sampler must still target the specified stationary distribution and its convergence must be justified. Independent chains and thinning do not themselves establish either property. For a reversible chain started at its stationary law, a valid procedure is the pivot construction of Besag and Clifford [2]: the observed graph is advanced L steps to a pivot, and the B comparison graphs are independent L-step runs from the pivot, which makes the observed and simulated graphs exchangeable under the null. The swap chain with uniformly chosen pairs of edges is symmetric and therefore reversible with respect to the uniform law on its reachable component. Any reported p-value should therefore be accompanied by the initialization, the proposal probabilities, the handling of rejected moves, the evidence for convergence, and the basis for the claimed calibration. The comparison replaces the assumptions of the theorems above by explicitly specified null-model and sampling assumptions, and it should be described in those terms.

A shadow-graph null supports conclusions about boundary-crossing structure. A full-graph reachability null must additionally specify what happens to internal edges and relation labels. Holding the internal edges fixed and rewiring only the crossing edges is a coherent choice, provided the preservation constraints stated above are made explicit.

1.4 What Is Reported

Three quantities are reported separately for a chosen entry vertex or component statistic T. The first is the absolute size of the reach set or component. The second is its fraction of the graph, since a component of twenty vertices can be exceptional in an enormous graph while occupying a negligible share of it. The third is its extremeness relative to the null, since a large reach set can be entirely typical under the null and remain operationally important. An upper-tail result establishes unusual reachability relative to the null, which is different from macroscopic reachability. The statistic K_{dir} is exactly constant across any null preserving in- and out-degrees, so it characterizes the background, while T tests the arrangement of the edges.

1.5 Witnesses

The path-level finding is a labelled walk in the full graph G from an entry vertex v to a vertex s in a designated sensitive set $S \subseteq V$, whose label sequence lies in a language $A \subseteq R^*$ of capability-preserving sequences. Taking A to be regular makes the search a product construction with a finite automaton and defines the searchable fragment of the problem. If the graph extraction and the accepted language overapproximate every execution relevant to the specified exposure, an actual exposure entails an accepted graph walk. Under this coverage assumption, such a walk is necessary but not sufficient for exposure, and a walk found this way should be called a candidate capability path until execution feasibility is established. Without the coverage assumption, failure to find a walk establishes only absence within the represented abstraction, since missing dynamic calls, omitted relations or an overly restrictive language can produce false negatives. A sequence of individually possible calls may require incompatible branch conditions, different runtime configurations, or permissions that cannot coexist, and a language over relation labels does not capture those state conditions. Comparison between two versions of the graph should consider changed labels, permissions and guards as well as newly introduced edges, since an existing edge can acquire a new security meaning without being new.

Two descriptive quantities accompany a witness. Disjointness is defined internally: two routes from an entry set to a sensitive set are vertex-disjoint when they share no vertex other than a common source or destination, and a non-endpoint vertex is essential when its removal eliminates every accepted route from the entry set to the sensitive set. If the search enforces the relation language through a product with an automaton, disjointness and essentiality are counted over the original program vertices and edges, because distinct automaton states attached to one program symbol must not make that symbol appear to be several independent resources. An essential vertex marks a place to inspect, and it does not establish that removing or restricting it is a valid repair.

The degree-moment ratios characterize branching propensity under specified random-graph models and provide contextual screening statistics. Operational exposure is assessed separately, through entry-specific forward reachability and candidate capability paths in the full relation-labelled graph that retains within-package transitions. Reachability statistics are reported with their absolute sizes, normalized sizes and, where the sampling procedure justifies it, comparisons against a constrained degree-preserving null ensemble. Statistical exceptionality prioritizes investigation, a candidate path identifies the relations requiring inspection, and a confirmed security finding additionally establishes that those relations compose under compatible execution conditions.

1.6 Spanning Connectivity Versus Path Quality in a Parallel Setting

A parallel distinction appears in the study of porous electrodes. Rodriguez and coauthors [1] reconstruct four electrode microstructures as three-dimensional binary volumes, locate the conducting component that spans the volume by a flood fill that starts on one face and tests whether the visited set reaches the opposite face, record the fraction P_∞ of conducting voxels belonging to that component, and then solve for the effective conductivity of the spanning component by finite volumes. All four volumes span, with P_∞ equal to 0.996, 0.999, 0.997 and 0.901, while the normalized conductivity $\sigma_{\mathrm{eff}}/\sigma_{\mathrm{bulk}}$ equals 0.306, 0.303, 0.706 and 0.134 respectively. Three of the four values of P_∞ lie within 0.003 of one another, yet the conductivities of those three differ by more than a factor of two. The authors conclude that spanning connectivity is necessary for conduction and that its magnitude is set by the quality of the spanning skeleton, through bottlenecks, tortuosity and alignment, which is the same division of roles proposed here between statistics of the large component and the properties of a particular path.

The correspondence is structural. A flood fill from an inlet face that tests arrival at an outlet face is a forward reach from an entry set with a test for membership of a target set, so the spanning test is the existence of a witness in the sense used above. The effective conductivity is a quantity that depends on how many routes exist and how narrow the narrowest cross-section is. A security analogue of that dependence is the number of edge-disjoint or vertex-disjoint paths from the entry set to the sensitive set, which by Menger's theorem equals the size of a minimum cut and is computable by a maximum-flow calculation, together with the vertices that every such path must pass through. A candidate capability path that runs through a single chokepoint is cheaper to remediate and may be more fragile than one with many independent routes, so the minimum cut is worth reporting beside the witness, though the analogy to conductivity is a guide to what to compute and does not make the two quantities equivalent.

The study should be cited for this qualitative point and with several cautions. The evidence consists of four morphologies, simulated without experimental validation, as the authors state, and the reported correlations are Pearson coefficients over four points. The source does not define σ_{bulk}, and the foam sample has a layer-wise conducting fraction of 0.186 with $\sigma_{\mathrm{eff}}/\sigma_{\mathrm{bulk}} = 0.706$, which would exceed the volume-fraction bound if the conducting fraction is the solid volume fraction and σ_{bulk} is the conductivity of the solid, so the normalization should be clarified before the values are used quantitatively. The bottleneck index is also defined in two ways, as the tenth percentile of the cross-sectional area along the axis normalized by its median in one section and as the reciprocal of the smallest normalized layer fraction in another, and the second grows as the thinnest layer becomes thinner, which is difficult to reconcile with its reported positive correlation with conductivity. The numbers support the claim that spanning does not fix transport, and they do not validate the statistics proposed in this note.

1.7 Design of a Study on a Pinned Real Graph

The first study on real software asks a specific question: which functions of a dependency have candidate call paths to operations that launch an external process. The sensitive set consists of the process-execution operations of the language and its standard library, and the vertices are functions identified by package, version and qualified name. Only possible-call edges are treated as propagation edges. Imports and general symbol references are recorded as contextual relations, since counting them as propagation would obscure what a path means. The study fixes a workspace, a lockfile, a target and a feature configuration, so that the graph is a function of a pinned build and can be regenerated. The full graph retains internal calls, and the shadow graph is derived from the calls that cross package boundaries. Calls that the extractor cannot resolve, including dynamic dispatch, calls through foreign interfaces and calls whose targets depend on conditional compilation or runtime values, are recorded as explicit unresolved edges, because their omission limits every negative finding, and a missing path is reported as absence within the extracted abstraction. Build-time code and code generation are treated as separate execution contexts and are not merged with application runtime paths.

Results are reported for each externally callable function of a dependency individually before any aggregation. A synthetic entry vertex connected to every function of a package would grant the attacker entry points that the application never exposes and would inflate the number of disjoint routes, so aggregation over a package is a summary of the individual results and replaces none of them. For each reported path the witness, the edge-disjoint and vertex-disjoint route counts and the essential vertices are given, and a small set of paths and chokepoints is inspected by hand to test whether the distinctions drawn on synthetic graphs survive actual call structure. Runtime tracing can confirm particular paths, although an unobserved path remains unresolved and is not thereby

disproved. The ecosystem is chosen by the tractability of extraction and by the relevance of the software to the study, since static resolution of calls is easier where package boundaries and name resolution are fixed at build time and harder where imports and call targets can change at runtime.

1.8 Implementation and Validation on Synthetic Graphs

The procedures above were implemented in plain Python using only the standard library and exercised on synthetic graphs, with four experiments addressing the threshold statistics, the irreducibility of the constrained chain, the calibration of the p-value procedure, and the power of a path-level statistic. The generator draws a directed graph in the style of the Chung–Lu model, with independent Pareto weights of exponent 2.5 for out- and in-propensity, assigns each vertex to one of a fixed number of packages, and draws the target of each edge from the source's own package with probability 0.4. The shadow graph keeps the edges whose endpoints lie in different packages. All results below come from one fixed seed per experiment, so they illustrate behaviour on this family of graphs and do not establish it for software dependency graphs.

Threshold Statistics. Table 1 compares the degree-moment ratios with the largest weakly and strongly connected components of the shadow graph for $n = 3000$ vertices and twenty packages, one graph per row. The undirected ratio K first exceeds 2 at the second row, where the largest weakly connected component holds 10.5% of the vertices, and that fraction rises to 53.9% in the next row and 93.9% at $K = 5.76$. The directed ratio K_{dir} crosses 1 between the fourth and fifth rows, where the largest strongly connected component is 0.3% and then 0.9% of the vertices, and that component occupies 33.3% at $K_{\mathrm{dir}} = 1.69$ and 62.2% at 2.35. The ratios therefore mark the onset of large components in the expected order and carry no information about their size, which depends on how far past the threshold the graph lies and is blurred at this n by finite-size effects and by the package structure of the generator.

Mean out-degree	Mean degree	K	K_{dir}	Largest WCC	Largest SCC
0.4	0.441	1.541	0.230	0.006	0.000
0.8	0.925	2.132	0.440	0.105	0.000
1.2	1.384	2.774	0.700	0.539	0.001
1.6	1.877	3.342	0.945	0.737	0.003
2.0	2.297	3.912	1.163	0.820	0.009
3.0	3.447	5.758	1.691	0.939	0.333
4.0	4.538	7.442	2.349	0.979	0.622

Table 1: Shadow graph statistics for $n = 3000$. Component sizes are fractions of the vertex set, and the mean out-degree refers to the full graph before internal edges are removed.

Irreducibility of the Constrained Chain. The question left open above was examined exhaustively on tiny instances. Six vertices in three packages of two vertices each allow 24 crossing arcs. For $m = 6$, 7 and 8 arcs, degree classes with between two and 3000 realizations were sampled at random (200, 150 and 200 classes respectively), every realization was enumerated, and the connected components of the move graph were computed for three move sets: swaps that keep every edge crossing a boundary (variant A), swaps restricted to edge pairs with the same pair of packages (variant B), and variant A with the addition of crossing-preserving reorientations of directed 3-cycles. Table 2 gives the number of degree classes in which the state space was connected.

The unrestricted swaps with the crossing constraint left the state space disconnected in 24%, 39% and 39% of the classes, adding 3-cycle reorientation reduced this to 8%, 12% and 13%, and variant B was disconnected in about 95% of the classes. The constrained chain is therefore not irreducible in general even with the additional move, the stricter conditioning of variant B makes it nearly useless as a sampler at this scale, and the results indicate that the reachable component of the observed graph is a real restriction to be reported and not a technicality.

Edges m	Classes	A connected	B connected	A with 3-cycles connected
6	200	152	10	184
7	150	91	7	132
8	200	123	9	174

Table 2: Degree classes with a connected state space, out of the classes tested, for three move sets on six vertices in three packages.

Calibration of the p-value Procedure. To test the sampler mechanics, the observed graph was taken to be a stationary draw from the chain after burn-in on a shadow graph with $n = 200$ vertices, eight packages and 315 edges, so that the null hypothesis holds by construction. The statistic was the forward reach of a randomly chosen vertex, and 1000 trials were run with $B = 39$ comparison graphs at nominal level 0.05, for which the binomial standard error is 0.007. Two procedures were compared at run lengths L equal to $0.25m$, m and $5m$ proposals. The naive procedure runs each comparison graph from the observed graph and rejected at rates 0.007, 0.034 and 0.043, so it is conservative at short run lengths and approaches the nominal level only slowly. The pivot procedure rejected at rates 0.051, 0.045 and 0.050, within about one standard error of nominal at every run length. This supports using the pivot construction, and it tests only the sampler on a graph that is a draw from the null, so it says nothing about whether the null matches a real graph.

Detection of a Planted Path. On forty synthetic graphs with $n = 1000$ vertices, twenty packages and about 1730 shadow edges, a path $v_0 \to x_1 \to x_2 \to s$ through four distinct packages was added, and two statistics were compared with the pivot null at $B = 199$ and $L = 5m$: the shortest-path distance from v_0 to s in the shadow graph, with small values extreme, and the forward reach of v_0, with large values extreme. The degree ratio K_{dir} was identical across null samples to six decimals (1.818287 in the first graph), as degree preservation requires, while the undirected K varied slightly (5.199 to 5.216 in the same graph) because the undirected projection merges opposite edges. The distance statistic rejected at level 0.05 in 34 of 40 planted instances, a power of 0.85, while the reach statistic rejected in 3 of 40, a power of 0.075. Among 1000 unplanted vertex pairs, 25 on each graph, the distance statistic rejected at rate 0.022 and the reach statistic at rate 0.025, both below the nominal 0.05. The planted instances that were not detected were graphs in which the null median distance was only 8 to 10, so that a three-edge path was not extreme enough, and the shortfall below nominal on unplanted pairs may reflect ties in the discrete distance statistic and the fact that the generator does not draw exactly from the null law, though neither explanation was tested. The graphs in this experiment already lie above both thresholds (mean shadow degree 3.45, $K_{\mathrm{dir}} \approx 1.7$), so a global reach statistic is uninformative and the path-level statistic carries the signal, which is the division of roles that the preceding sections argue for.

Minimum Cut Against Distance. The suggestion drawn from the electrode study, to report the number of independent routes beside the witness, was tested with an entry set of three vertices

in one package and a sensitive set of three vertices in another. The statistics were the distance from the entry set to the sensitive set, with small values extreme, and the maximum number of edge-disjoint paths between the sets, computed by unit-capacity maximum flow and capped at 10, with large values extreme. For each of 40 graphs with $n = 1000$ and twenty packages, 8 unplanted pairs of sets (320 in all) gave the false-positive rate, one narrow plant gave a single three-edge path from an entry vertex to a sensitive vertex, and one wide plant gave three vertex-disjoint three-edge paths. The pivot null used $B = 99$ and $L = 5m$. In a sparse regime with mean out-degree 0.8, the false-positive rate was 0.003 for both statistics, since both observed and null values were almost always the same trivial value. The narrow plant was detected by the distance statistic in 92.5% of graphs and by the minimum cut in 90.0%, with an observed minimum cut of exactly 1 in all 40 graphs, and the wide plant was detected by the distance statistic in only 20.0% of graphs but by the minimum cut in 100%, with an observed minimum cut of exactly 3 in all 40. The weak distance power for the wide plant is not an artefact of the code: in six graphs checked separately, the null probability of a distance of at most three between the sets ranged from 0.01 to 0.13, because three entry edges and three sink edges make a short chance path much more likely than for a single pair, while three independent routes remain rare under the null. The minimum cut therefore both detects wide structure that distance misses and, as a descriptive quantity, separates a single chokepoint (1) from several routes (3) in the planted cases. In a dense regime with mean out-degree 3.0, past both thresholds, the picture degraded: the distance statistic rejected in 15.0% of narrow plants and 2.5% of wide plants, the minimum cut in 0% and 15.0%, and the false-positive rates were 0.041 and 0.013. The observed minimum cut for narrow plants ranged from 1 to 6 across graphs, because background routes already supplied several independent paths between the sets. A minimum cut therefore describes planted structure only when the background is sparse, and in a graph already past the thresholds it chiefly reflects the degrees of the entry and sensitive sets.

Chokepoint Vertices. Edge-disjoint routes can share a vertex, so the edge-disjoint count can overstate how many independent routes exist when they converge on one symbol. A further run in the sparse regime, a separate random draw with the same settings as above, added a third plant, the funnel, in which three routes from the entry vertices pass through one hub vertex before reaching the sensitive vertices, so that there are three edge-disjoint paths but only one vertex-disjoint path. Three statistics were compared: the distance, the edge-disjoint count, and the vertex-disjoint count obtained by splitting each vertex and giving it unit capacity. For each observed graph the essential vertices, those outside the two sets that every route from the entry set to the sensitive set must cross, were also listed by removing each candidate in turn. Over 320 unplanted pairs all three statistics had a false-positive rate of 0.003. The narrow plant was detected in 87.5% of graphs by each statistic. The wide plant was detected in 30.0% of graphs by distance and in 100% by both counts. The funnel plant was detected in 15.0% of graphs by distance, in 100% by the edge-disjoint count and in only 15.0% by the vertex-disjoint count. The observed values were those intended: the narrow plant gave a vertex-disjoint count of 1 in every graph, the wide plant gave 3, and the funnel gave an edge-disjoint count of 3 with a vertex-disjoint count of 1 in all graphs but one pair, where a background route raised it to 2. The hub was listed among the essential vertices in 39 of the 40 funnel graphs. The edge-disjoint count was thus the better test statistic for these sparse plants, and the conclusion is limited to them. The low vertex-based detection rate for the funnel is compatible with that statistic doing its intended descriptive job, since three routes sharing a hub provide only one internally vertex-disjoint route. One possible reason for the weak test power is that a single route through a high-degree vertex is not unusual under a null that preserves every vertex's degrees, and this explanation was not tested. The vertex-disjoint count and the essential-vertex list supplied

the description that the edge count could not, namely that the three apparent routes depend on one symbol. The hub identifies a location to inspect, and it does not prove that removing or restricting it is a valid repair. This run used the sparse regime only. The false-positive count was one pair out of 320 for each statistic, and the 320 pairs come from 8 pairs in each of 40 graphs, so the pairs within a graph share the same null samples and are not independent, which widens the uncertainty of the rate beyond what a binomial calculation on 320 trials would suggest. The same applies to the false-positive rates reported for the preceding run, which were 1 of 320 for both statistics in the sparse regime and 13 of 320 for distance and 4 of 320 for the minimum cut in the dense regime.

Limits of the Validation. The planted path is the easiest case, with three edges, uniformly random packages and no camouflage, and heavier hubs, adversarial placement, and the effect of the within-package structure were not examined. The vertex-disjoint count and the essential-vertex list were evaluated only in the sparse regime, where the background supplies few routes of its own. Variant B was not used in the detection experiment because of its disconnected state space. The graphs have 10^2 to 10^3 vertices and come from a synthetic generator, so the next step is a concrete graph extractor and relation semantics applied to a real dependency graph, followed by the same calibration check on the extracted graph's reachable component.

1.9 Pilot on Two Pinned Python Projects

The design above was applied to two sets of pinned distributions downloaded as wheels from the Python Package Index and parsed with the standard `ast` module without being executed. The first project takes `virtualenv` 20.26.3 as the application, with `distlib` 0.3.8, `filelock` 3.15.4 and `platformdirs` 4.2.2 as dependencies, and the second takes `GitPython` 3.1.43 as the application, with `gitdb` 4.0.11 and `smmap` 5.0.1 as dependencies. The first project has 121 modules, 1339 functions and 197 classes, with 525 externally callable functions in its dependencies, and the second has 72 modules, 1045 functions and 149 classes, with 293 such functions. No file failed to parse. The entries are the public functions and methods of the dependencies, taken one at a time, and the sensitive set contains the process-launching functions of `subprocess`, the `os` functions `system`, `popen`, the `exec` and `spawn` families and `posix_spawn`, `pty.spawn`, and the asynchronous subprocess constructors.

Edges are possible calls between functions, resolved statically through direct names, import aliases, methods called on the first parameter of a method, constructors, and calls through `super` to analyzed base classes. Calls on receivers of unknown type are recorded as unresolved. Three graphs were built from the same resolution. The strict graph keeps resolved calls only. The matched graph also links each call on an unknown receiver to every analyzed method of the same name, provided there are at most ten. The filtered graph does the same but only for methods in the caller's own distribution and only for names that are not attributes of the built-in types or of common standard-library objects. Code at module level is kept as a separate import-time vertex that nothing calls, and no import-time call to a process launcher was found in either project. A search of the source found 3 direct launch sites in `distlib`, 7 in `virtualenv` and 3 in `GitPython`, none in the other distributions, and the extractor found the same numbers, which checks the recognition of launch calls and does not check the recall of paths. The extraction leaves many calls unresolved: 1315 in `distlib`, against 498 resolved internal calls, 414 in `gitdb` and 285 in `smmap`.

In the strict graphs the shadow graphs are tiny. The first has 16 crossing edges among 23 vertices, with $K = 2.88$ and $K_{\mathrm{dir}} = 0$, and the second has 32 crossing edges among 47 vertices, with $K = 1.78$ and $K_{\mathrm{dir}} = 0$, because no vertex has both an incoming and an outgoing crossing edge. The degree-moment screen therefore has nothing to say about these graphs, and the entry-level witnesses are the finding. In the first project 6 of the 525 entries have a witness in the strict graph,

all in `distlib`, and in the second project none of the 293 do. Table 3 lists the six, each with an edge-disjoint and a vertex-disjoint count of one. Every route was checked against the source and matches it. The call from `upload_file` to `sign_file` lies inside a branch that requires a signer argument and an available signing program, so that route is a candidate capability path whose feasibility depends on conditions the graph does not carry, which is the distinction the note draws between a graph witness and an execution witness. The filtered graph gives the same six witnesses in the first project and none in the second.

Entry in `distlib`	Route to a launcher	Edge	Vertex	Essential vertices
`SubprocessMixin.run_command`	direct call to `Popen`	1	1	none
`PackageIndex.__init__`	direct call to `check_call`	1	1	none
`PackageIndex.run_command`	direct call to `Popen`	1	1	none
`PackageIndex.sign_file`	through `run_command`	1	1	`run_command`
`PackageIndex.upload_file`	through `sign_file`	1	1	`sign_file`, `run_command`
`PackageIndex.verify_signature`	through `run_command`	1	1	`run_command`

Table 3: Entries with a witness in the strict graph of the first project, with edge-disjoint and vertex-disjoint route counts and the vertices every route must cross.

The matched graph shows how sensitive the structural screen is to extraction choices. In the first project it has 865 crossing edges among 324 vertices, with $K = 11.87$ and $K_{\mathrm{dir}} = 1.11$, which places it past both thresholds, and 207 entries have witnesses (193 in `distlib`, 12 in `platformdirs` and 2 in `filelock`) along routes of 1 to 12 edges, with a route count above one for only 5 of them. In the second project it has 47 entries with witnesses (38 in `gitdb` and 9 in `smmap`). One of the shortest of these begins with a `write` method in `gitdb` whose call to `update` on a hash object is matched by name to an unrelated `update` method in `GitPython`, which was confirmed in the source to be a name collision. The matched graph is thus an upper bound on what an unresolved call could hide, dominated here by collisions of common method names, and the filtered graph removes them at the cost of missing any genuine call that passes through a built-in method name. Negative findings, including the absence of witnesses in `gitdb` and `smmap`, hold only within the strict and filtered abstractions and are bounded by the unresolved calls reported above.

These results are a pilot. They cover two small sets of pure-Python distributions, no null comparison was run because the strict shadow graphs are too small to support one, calls through callbacks, thread targets and attribute lookup at runtime are not represented, and no runtime trace was used to confirm or refute a path. They do show that the synthetic distinctions have counterparts in real call structure: a witness with a chokepoint vertex that is a single place to inspect, a route that depends on conditions outside the graph, and a structural screen whose verdict changes with the treatment of unresolved calls.

References

[1] A. Rodriguez, J. Ortegón, A. Rios, C. Couder, and R. Barbosa. Three-Dimensional Infinite Cluster Function as a Descriptor of Through-Plane Effective Conductivity in Porous Electrodes of Membrane Electrode Assemblies. *Materials*, 19(5):835, 2026.

[2] J. Besag and P. Clifford. Generalized Monte Carlo Significance Tests. *Biometrika*, 76(4):633–642, 1989.

[3] M. Molloy and B. Reed. A Critical Point for Random Graphs with a Given Degree Sequence. *Random Structures and Algorithms*, 6(2–3):161–180, 1995.

[4] S. N. Dorogovtsev, J. F. F. Mendes, and A. N. Samukhin. Giant Strongly Connected Component of Directed Networks. *Physical Review E*, 64:025101, 2001.

[5] C. Cooper and A. Frieze. The Size of the Largest Strongly Connected Component of a Random Digraph with a Given Degree Sequence. *Combinatorics, Probability and Computing*, 13(3):319–337, 2004.

[6] A. R. Rao, R. Jana, and S. Bandyopadhyay. A Markov Chain Monte Carlo Method for Generating Random $(0, 1)$-Matrices with Given Marginals. *Sankhyā: The Indian Journal of Statistics, Series A*, 58(2):225–242, 1996.

[7] A. Berger and M. Müller-Hannemann. Uniform Sampling of Undirected and Directed Graphs with a Fixed Degree Sequence. arXiv:0912.0685, 2010.

[8] P. Erdős and A. Rényi. On the Evolution of Random Graphs. *Publications of the Mathematical Institute of the Hungarian Academy of Sciences*, 5:17–61, 1960.